

ME2605 / DDA6204 · Fall 2026

Lecture 4

Normalization II



Parameters, representation power, and gradients

Ruoyu Sun

Lecture 4: Normalization II

Part 1 · Layer normalization

- 1.1 Number of learnable parameters
- 1.2 Representation power
- 1.3 Gradients of LayerNorm

Part 2 · Batch normalization

- 2.1 Introduction to BatchNorm
- 2.2 Training–inference consistency

Questions for this lecture

- LayerNorm: how many parameters do we learn for d features and B samples?
- BatchNorm: can other samples change the prediction for the same input?
- How should BatchNorm predict when only one test sample arrives?

Part 1

Layer normalization

Review: the MLP network expression

MLP (multi-layer perceptron), also called a fully connected network (FCN).

This is the network architecture we have used so far.

$$z_0 = x,$$

$$h_l = W_l z_{l-1} + b_l, \quad l = 1, \dots, L, \quad (\text{pre-activation})$$

$$z_l = \phi(h_l), \quad l = 1, \dots, L - 1, \quad (\text{post-activation})$$

$$f_\theta(x) = h_L, \quad \theta = \{W_l, b_l\}_{l=1}^L.$$

$$x \in \mathbb{R}^{d_0}, \quad W_l \in \mathbb{R}^{d_l \times d_{l-1}}, \quad b_l \in \mathbb{R}^{d_l}, \quad h_l, z_l \in \mathbb{R}^{d_l}.$$

L includes the output layer: there are $L - 1$ hidden layers.

In the first segment of the course, we discuss MLPs.

We will discuss Transformers later.

LayerNorm: a differentiable layer

- We have already introduced LayerNorm

A transformation of the layer output

- View the standardization step as a nonlinear transformation and add it to neural networks
- It plays a similar role to the activation function and other layers: a layer
- The operation is differentiable; compatible with BackProp

First review one sample, then count the learned parameters.

Part 1.1

Number of learnable parameters in LayerNorm

Normalization operators: scalar and vector parameters

One sample $z \in \mathbb{R}^d$. Both operators use the same computed statistics.

1. Compute statistics: μ is the mean and σ is the standard deviation.

$$\mu(z) = \frac{1}{d} \sum_{j=1}^d z_j, \quad \sigma^2(z) = \frac{1}{d} \sum_{j=1}^d (z_j - \mu(z))^2$$

2. Standardize the feature vector

$$\hat{z} = \frac{z - \mu(z)\mathbf{1}}{\sqrt{\sigma^2(z) + \varepsilon}}, \quad \varepsilon > 0$$

3. Apply a learnable affine transformation: γ and β are its parameters.

NO-S: scalar gain and bias

$$\tilde{z} = \text{NO-S}_{\gamma, \beta}(z) := \gamma \hat{z} + \beta \mathbf{1}$$

$$\gamma, \beta \in \mathbb{R} \quad (2 \text{ learned scalars})$$

Same gain and bias for every feature.

NO-V: featurewise gain and bias

$$\tilde{z} = \text{NO-V}_{\gamma, \beta}(z) := \gamma \odot \hat{z} + \beta$$

$$\gamma, \beta \in \mathbb{R}^d \quad (2d \text{ learned scalars})$$

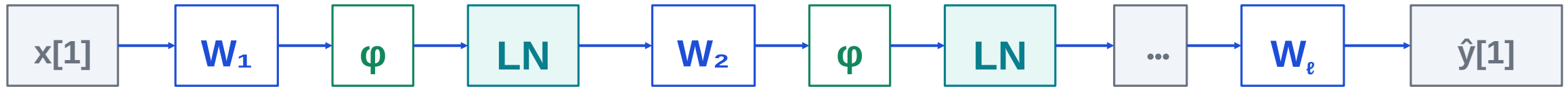
One gain and bias per feature.

Standard LN uses NO-V, with learned γ and β shared across samples.

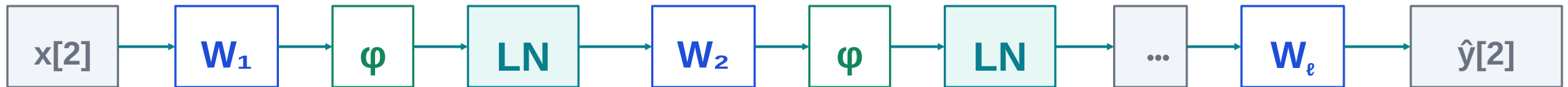
LayerNorm: one network, multiple samples

A mini-batch of $B = 3$ samples passes through the same network in parallel.

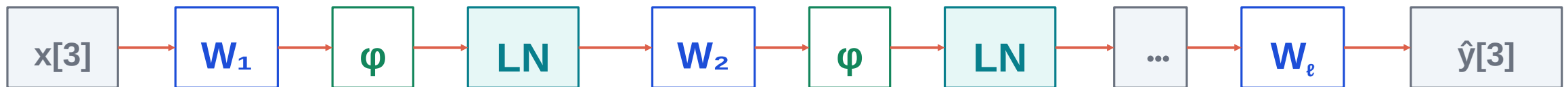
Sample 1



Sample 2



Sample 3



$$z_\ell = \phi(W_\ell \tilde{z}_{\ell-1} + b_\ell) \quad \tilde{z}_\ell = \text{LN}_{\gamma_\ell, \beta_\ell}(z_\ell) = \text{NO-V}_{\gamma_\ell, \beta_\ell}(z_\ell)$$

Shared at each layer: W_ℓ , b_ℓ , γ_ℓ and β_ℓ . Each sample computes its own μ and σ .

W boxes include biases; $\tilde{z}_0 = x$. Next, collect the three z_1 vectors into Z_1 .

LayerNorm on a 6×3 activation matrix

Select $B = 3$ samples from N training samples; hidden layer 1 has $d_1 = 6$ neurons.

$$\mathbf{Z}_1 = [\mathbf{z}_1[1], \mathbf{z}_1[2], \mathbf{z}_1[3]] \in \mathbb{R}^{6 \times 3}$$

	Sample 1	Sample 2	Sample 3
Feature 1	1	3	6
Feature 2	2	2	2
Feature 3	0	1	5
Feature 4	4	6	1
Feature 5	5	2	3
Feature 6	1	0	1

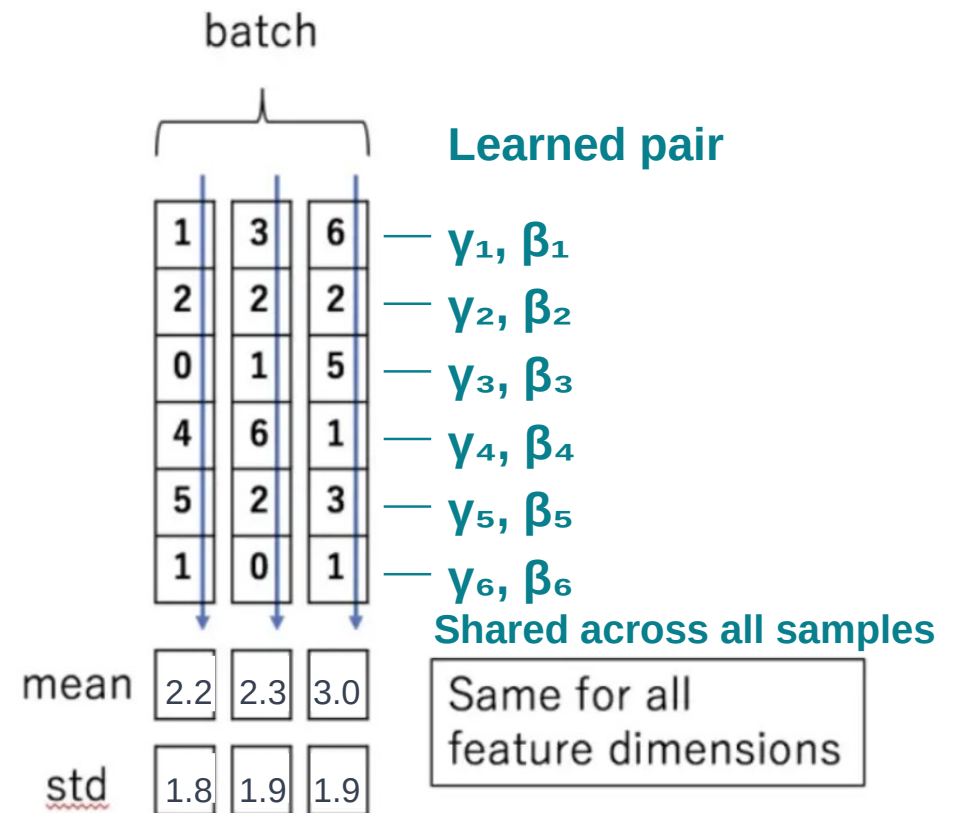
6 rows: neurons in hidden layer 1.

3 columns: samples in this mini-batch.

Each column has its own μ and σ .

Each row shares one γ_j, β_j pair.

Layer Normalization



How many statistics and parameters?

One LN layer has d features per sample and processes B samples.

Features d	Samples B	Computed μ and σ	NO-S: learned γ and β	NO-V: learned γ and β
5	1	1 of each	1 γ & 1 β (2)	5 γ & 5 β (10)
5	20	[1] ____ of each	[2] ____ total	5 γ & 5 β (10)
10	20	20 of each	1 γ & 1 β (2)	[3] ____ total

- Vote 1: blank [1]

A. 5 B. 20 C. 40
- Vote 2: blank [2]

A. 2 B. 10 C. 40
- Vote 3: blank [3]

A. 10 B. 20 C. 40

[1] counts μ and σ separately. [2] and [3] count learned scalars in total.

LayerNorm: statistics versus learned parameters

The answers: [1] 20 of each statistic, [2] 2 learned scalars, [3] 20 learned scalars.

Features d	Samples B	Computed μ and σ	NO-S: learned γ and β	NO-V: learned γ and β
5	1	1 of each	1 γ & 1 β (2)	5 γ & 5 β (10)
5	20	20 of each	1 γ & 1 β (2)	5 γ & 5 β (10)
10	20	20 of each	1 γ & 1 β (2)	10 γ & 10 β (20)
d	B	B of each	1 γ & 1 β (2)	d γ & d β ($2d$)

Each sample computes its own mean and standard deviation.

All samples share the learned γ and β of this layer.

Changing B does not change the number of learned parameters.

Practice questions: how many LN parameters?

A mini-batch has $B = 5$ samples.
Four hidden layers each have 8 neurons and one separate LN module.

- 1. What is the shape of Z_3 , the third hidden layer's post-activation matrix?
- 2. How many learned γ and β values are there?

Affine convention	Third hidden layer	All four hidden layers
Shared scalar (simplified)	? γ + ? β	? γ + ? β
Featurewise (standard LN)	? γ + ? β	? γ + ? β

- 3. If B increases from 5 to 10, which parameter counts change?

Count LN parameters only, excluding the weights W_e and biases b_e .

Part 1.2

Representation power

Why discuss representation power?

We have counted γ and β . What can these parameters change?

1. Do vector γ and β offer more than a single scalar pair?
2. Can γ and β recover the original network's representation power?

Lecture 3: equality of attainable output sets

Fix d and $\varepsilon > 0$, and allow the module input z to range over $\mathbb{R}^d$.

Proposition: the full attainable output set remains $\mathbb{R}^d$

$$\{\text{NO-}S_{\gamma,\beta}(z) : z \in \mathbb{R}^d, \gamma > 0, \beta \in \mathbb{R}\} = \{z : z \in \mathbb{R}^d\} = \mathbb{R}^d$$

Inside this set, z , γ and β all vary.

Proof: every target z has a corresponding restoring pair

$$\forall z \in \mathbb{R}^d \exists \gamma(z) > 0, \beta(z) \in \mathbb{R} : \text{NO-}S_{\gamma(z),\beta(z)}(z) = z$$

$$\gamma(z) = \sqrt{\sigma^2(z) + \varepsilon}, \quad \beta(z) = \mu(z)$$

$$\text{NO-}S_{\gamma(z),\beta(z)}(z) = \sqrt{\sigma^2(z) + \varepsilon} \frac{z - \mu(z)\mathbf{1}}{\sqrt{\sigma^2(z) + \varepsilon}} + \mu(z)\mathbf{1} = z$$

The choices $\gamma(z)$ and $\beta(z)$ may vary with z .

This does not supply one parameter pair shared by all inputs.

One shared scalar pair cannot restore every input

Claim: one scalar pair (γ , β) cannot restore these three input samples.

Set $\varepsilon = 0$ for this calculation with three nonconstant vectors.

$$\nexists \gamma, \beta \in \mathbb{R} : \quad \text{NO-}S_{\gamma, \beta}(z^{[i]}) = z^{[i]} \quad \text{for all } i = 1, 2, 3$$

Module input	Restoring $\gamma(z)$	Restoring $\beta(z)$
$z[1] = (1, 2, 6)$	$\sqrt{\frac{14}{3}}$	3
$z[2] = (2, 5, 8)$	$\sqrt{6}$	5
$z[3] = (4, 5, 12)$	$\sqrt{\frac{38}{3}}$	7

Exact recovery of each nonconstant vector requires

$$\gamma(z) = \sigma(z), \quad \beta(z) = \mu(z)$$

One shared β cannot equal 3, 5 and 7 at the same time.

Representation power and training loss

Assume noiseless labels generated by a target function f^* .

Fix the architecture and parameter dimensions, but allow θ to vary.

$$y_i = f^*(x_i), \quad i = 1, \dots, n, \quad \mathcal{F} = \{ f_\theta : \theta \in \Theta \}$$

Sufficient for this target: one parameter setting matches f^* at every input.

$$f^* \in \mathcal{F} \iff \exists \theta^* \in \Theta : f_{\theta^*}(x) = f^*(x) \quad \forall x$$

Use mean squared error on the n training samples:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \|f_\theta(x_i) - y_i\|_2^2 \geq 0$$

$$L(\theta^*) = \frac{1}{n} \sum_{i=1}^n \|f^*(x_i) - f^*(x_i)\|_2^2 = 0$$

Sufficient representation gives zero training MSE

$$\min_{\theta \in \Theta} L(\theta) = 0$$

This is an existence statement, not a guarantee for an optimizer.

Discussion: does binary classification loss have minimum 0?

0–1 loss: yes for a perfect classifier. Sigmoid cross-entropy: 0 may be approached, but not attained.

Zero training MSE does not imply an exact target function

Can a model fit every training point without representing f^* on all inputs?

$$\mathcal{F} = \{ f_c(x) = c : c \in \mathbb{R} \}, \quad f^*(x) = \sin(\pi x)$$



Case 1: match f^* at every x

Requires f^* to belong to the class.

Case 2: match the training set

Only the five sample values must match.

Here Case 2 holds, but Case 1 fails.

$$\min_c L(c) = L(0) = 0, \quad f^* \notin \mathcal{F}$$

Lecture 1's empirical representation error cannot distinguish these two cases.

Definition: equal function classes

To understand representation power in more depth, we discuss function classes.

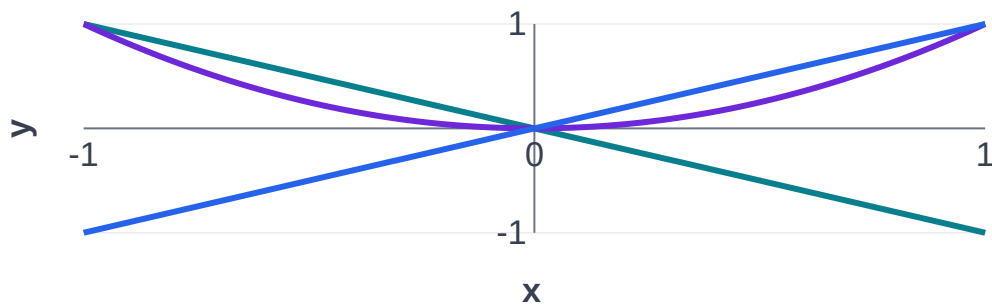
Function class: a set of functions.

Two function classes are the same if the two sets are the same.

Every function in either class must also appear in the other class.

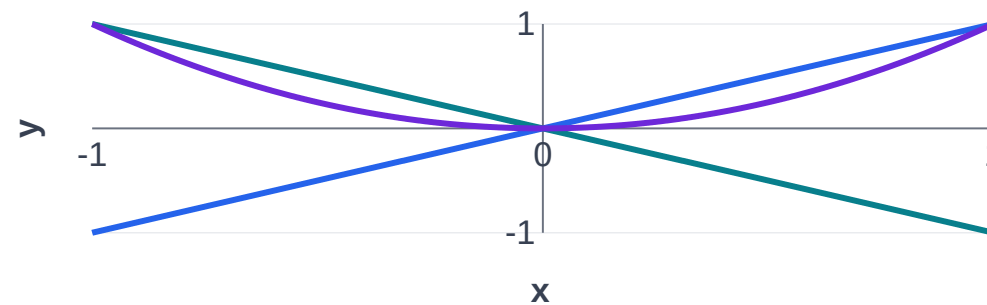
Here is a simple example on $x \in [-1, 1]$.

Class $F = \{f_1, f_2, f_3\}$



$$\text{---} f_1(x) = x \quad \text{---} f_2(x) = x^2 \quad \text{---} f_3(x) = -x$$

Class $G = \{g_1, g_2, g_3\}$



$$\text{---} g_1(x) = x^2 \quad \text{---} g_2(x) = -x \quad \text{---} g_3(x) = x$$

$$f_1 = g_3, \quad f_2 = g_1, \quad f_3 = g_2 \text{ on all of } [-1, 1] \quad \Rightarrow \quad \mathcal{F} = \mathcal{G}$$

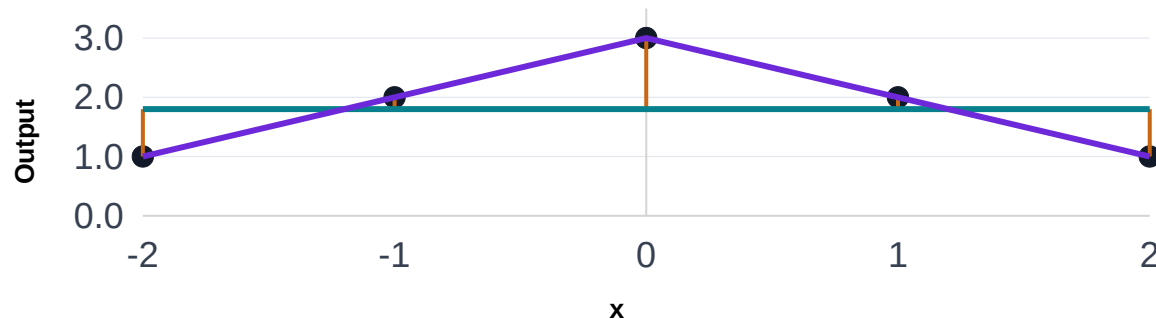
Best training loss: distance to a function class

In previous pages, we discussed perfect fit (the realizable case).

In this page, we discuss imperfect fit.

$$L_{\mathcal{F}}^* = \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \|f(x_i) - f^*(x_i)\|_2^2$$

Example: constant models



Purple: target Teal: best constant

Orange segments: residuals at the five samples

$$\mathcal{F} = \{f_c(x) = c : c \in \mathbb{R}\}$$

$$c^* = \frac{1 + 2 + 3 + 2 + 1}{5} = 1.8$$

Average the squared residuals:

$$L_{\mathcal{F}}^* = \frac{0.8^2 + 0.2^2 + 1.2^2 + 0.2^2 + 0.8^2}{5} = 0.56$$

Claim: equal classes have equal best training loss, and a larger class cannot increase this loss.

$$\mathcal{F} = \mathcal{G} \implies L_{\mathcal{F}}^* = L_{\mathcal{G}}^*, \quad \mathcal{F} \subseteq \mathcal{G} \implies L_{\mathcal{G}}^* \leq L_{\mathcal{F}}^*$$

Simple Examples on Representation Power

We use a few examples to help understand representation power.

No biases, with input $x \in \mathbb{R}^2$ and output $y \in \mathbb{R}^2$.

One layer: $y = Wx$ Five linear layers: $y = W_5W_4W_3W_2W_1x$.

1. Every hidden layer has width 2

All five weight matrices are 2×2 .

$$W_1 = W, \quad W_2 = W_3 = W_4 = W_5 = I_2,$$

$$W_5W_4W_3W_2W_1 = W.$$

Same function class as $y = Wx$.

2. A one-neuron bottleneck

Widths: $2 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 2$.

$$\text{rank}(W_5W_4W_3W_2W_1) \leq 1.$$

$$I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \text{rank}(I_2) = 2.$$

Cannot represent the identity $y = x$.

General case: equal linear function classes iff every hidden $d_\ell \geq \min(d_{\text{in}}, d_{\text{out}})$.

Next: can adding ReLU represent a function that no linear network can represent?

ReLU and linear networks represent different functions

Input and output are $\mathbb{R}^2$, the hidden width is 2, and there are no biases.

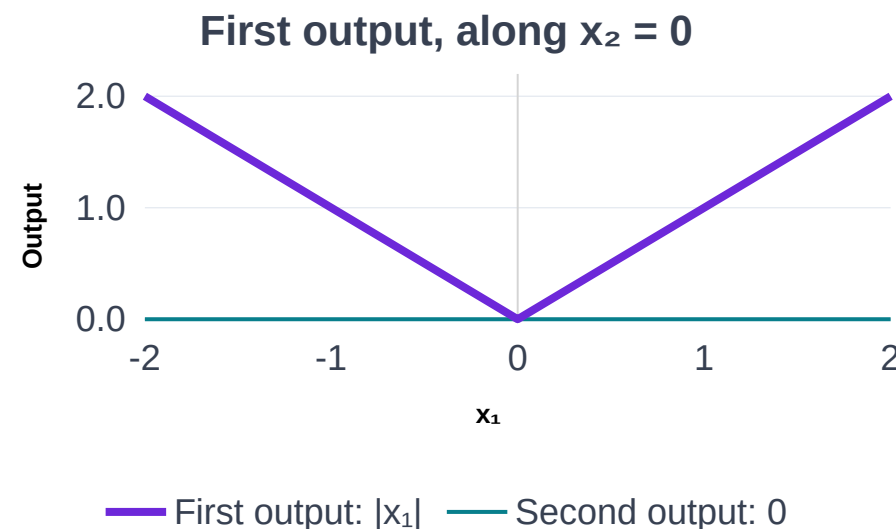
$$f_{\text{lin}}(x) = W_2 W_1 x, \quad f_{\text{ReLU}}(x) = W_2 \text{ReLU}(W_1 x), \quad W_1, W_2 \in \mathbb{R}^{2 \times 2}$$

Which network can represent $f^*(x) = (|x_1|, 0)$ on all of $\mathbb{R}^2$?

ReLU: two neurons are enough

$$W_1 = \begin{pmatrix} 1 & 0 \\ -1 & 0 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

$$W_2 \text{ReLU}(W_1 x) = \begin{pmatrix} \max(x_1, 0) + \max(-x_1, 0) \\ 0 \end{pmatrix} = \begin{pmatrix} |x_1| \\ 0 \end{pmatrix}$$



Linear networks cannot produce the change of slope in $|x_1|$.

$$A(-e_1) = -Ae_1, \quad f^*(-e_1) = f^*(e_1) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

With $A = W_2 W_1$ and $e_1 = (1, 0)$, matching $f^*(e_1)$ forces $A(-e_1) = (-1, 0)$, not $(1, 0)$.

LN before an affine layer: do γ and β increase representation power?

A somewhat surprising finding: γ and β may not necessarily increase representation power.

Let's consider a 1-hidden-layer NN with fixed widths d_0 , d_1 and d_2 .

$$W_1 \in \mathbb{R}^{d_1 \times d_0}, \quad W_2 \in \mathbb{R}^{d_2 \times d_1}, \quad b_1, \gamma, \beta \in \mathbb{R}^{d_1}, \quad b_2 \in \mathbb{R}^{d_2}.$$

Parameter ranges are omitted below; all trainable parameters vary and $\varepsilon > 0$ is fixed.

A. Original MLP $\mathcal{F}_A = \{x \mapsto W_2 \phi(W_1 x + b_1) + b_2\}$

B. LN without learned affine $\mathcal{F}_B = \{x \mapsto W_2 \text{LN}_{1,0}(\phi(W_1 x + b_1)) + b_2\}$
 $\gamma = 1, \beta = 0$ fixed

C. LN with learned affine $\mathcal{F}_C = \{x \mapsto W_2 \text{LN}_{\gamma,\beta}(\phi(W_1 x + b_1)) + b_2\}$

Proposition: B and C represent the same functions

$$\mathcal{F}_B = \mathcal{F}_C$$

In this MLP, an immediately following affine layer absorbs γ and β .

$$W'_2 = W_2 \text{diag}(\gamma), \quad b'_2 = W_2 \beta + b_2.$$

W_2 and b_2 can vary freely, so the next affine layer can absorb γ and β .

This proves B = C, not equality with the original network class A.

Equal function classes can have different GD updates

If γ and β can be absorbed, can they still affect training?

$$f_c(x) = cx, \quad g_{a,b}(x) = abx, \quad c = ab$$

Train on $x = 1, y = 2$ with loss $\frac{1}{2}(f(1) - 2)^2$ and step size 0.1.

$$\frac{\partial L}{\partial a} = b \frac{\partial L}{\partial c}, \quad \frac{\partial L}{\partial b} = a \frac{\partial L}{\partial c}$$

Model	Initial parameters / prediction	After one simultaneous GD step
Direct: $c x$	$c = 1$ / $f(1) = 1$	$c = 1.1$ / $f(1) = 1.1$
Factorized: $a b x$	$a = b = 1$ / $g(1) = 1$	$a = b = 1.1$ / $g(1) = 1.21$

The same function class can have different GD trajectories.

This calculation does not prove that extra parameters always speed up training.

Next: what changes when a nonlinear activation follows LN?

Pre-activation LN: affine parameters can add functions

An immediately following affine layer can absorb LN's γ and β .

For pre-activation LN, ϕ intervenes, so the function class can be larger.

$$\Theta = \mathbb{R}^{d \times p} \times \mathbb{R}^d, \quad \mathcal{A} = \mathbb{R}^{m \times d} \times \mathbb{R}^m, \quad \gamma, \beta \in \mathbb{R}^d$$

Without the learned affine step

$$\mathcal{G}_0 = \{x \mapsto W_2 \phi(\text{LN}_0(W_1 x + b_1)) + b_2 : (W_1, b_1) \in \Theta, (W_2, b_2) \in \mathcal{A}\}$$

With featurewise γ and β

$$\mathcal{G}_1 = \{x \mapsto W_2 \phi(\text{LN}_{\gamma, \beta}(W_1 x + b_1)) + b_2 : (W_1, b_1) \in \Theta, (W_2, b_2) \in \mathcal{A}, \gamma, \beta \in \mathbb{R}^d\}$$

Claim [informal]: the second class can be strictly larger

$$\mathcal{G}_0 \subseteq \mathcal{G}_1 \quad (\gamma = \mathbf{1}, \beta = \mathbf{0}), \quad \mathcal{G}_0 \subsetneq \mathcal{G}_1 \text{ can occur}$$

- The next ReLU example proves strict inclusion with both W_1 and W_2 trainable.

Strict inclusion is not asserted for every activation or architecture.

A ReLU example with $\varepsilon = 0$

[Reading]

[Reading] means optional material that you do not need to master.

Scalar input $x \in \mathbb{R}$, three hidden neurons, scalar output, and trainable biases.
In both classes, require $\sigma(x) > 0$ for every x so LN is defined on all of $\mathbb{R}$.

Without γ, β : at most three kinks for any W_1, W_2 and biases

$$\text{ReLU}\left(\frac{a_j x + t_j}{\sigma(x)}\right) = \frac{\text{ReLU}(a_j x + t_j)}{\sigma(x)}, \quad j = 1, 2, 3.$$

Each centered coordinate is affine in x and can cross zero at most once.

With γ, β : one network has four kinks

$$h(x) = W_1 x + b_1 = (x, 1, -x - 1)^\top, \quad D(x) = x^2 + x + 1 > 0.$$

$$\gamma = -\sqrt{\frac{2}{3}}(1, 0, 1)^\top, \quad \beta = -\frac{3}{\sqrt{7}}(1, 0, 1)^\top, \quad W_2 = (1, 0, 1), \quad b_2 = 0.$$

$$g_*(x) = \text{ReLU}\left(\frac{-x}{\sqrt{D(x)}} - \frac{3}{\sqrt{7}}\right) + \text{ReLU}\left(\frac{x+1}{\sqrt{D(x)}} - \frac{3}{\sqrt{7}}\right).$$

Four kinks: $-3, -1.5, 0.5, 2 \implies g_* \in \mathcal{G}_1 \setminus \mathcal{G}_0, \quad \mathcal{G}_0 \subsetneq \mathcal{G}_1.$

Learnable parameters may not recover the original class

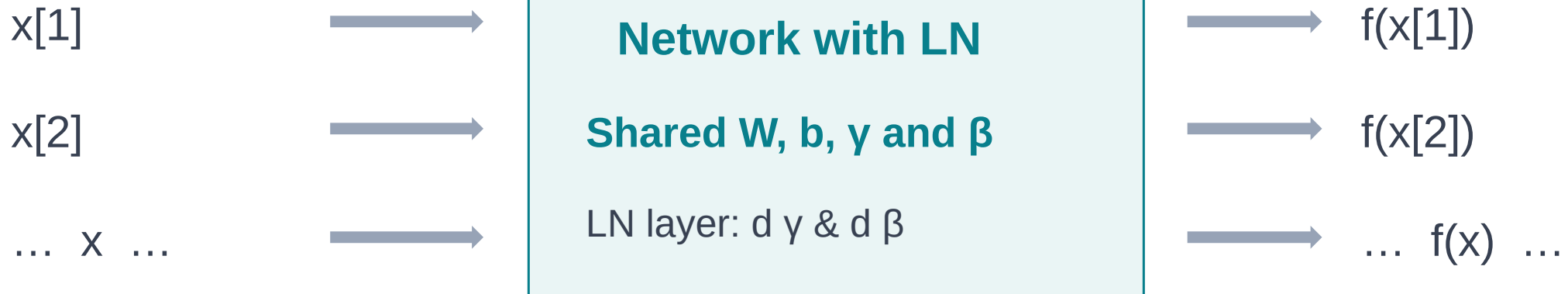
Question: can learned γ and β restore every function in the original MLP class?

For each $f \in \mathcal{F}$, find one g_η such that $g_\eta(x) = f(x)$ for every x .

Infinitely many possible inputs

One shared parameter set η

Match f at every input



A recovery choice that works for one input may fail on another.

We cannot choose fresh $\gamma(x)$ and $\beta(x)$ for each input in a standard LN layer.

Parameter counts alone are not a proof: the next optional example proves non-recovery.

A smaller function class on a bounded input domain

[Reading]

Provide an example where the network with LN represents a smaller function class.

Scalar input $x \in [0, 1]$, one hidden neuron, ReLU, and any fixed $\varepsilon > 0$.

Original network

$$\mathcal{F} = \{x \mapsto a \operatorname{ReLU}(wx + b) + c : a, w, b, c \in \mathbb{R}\}$$

Network with LN

$$\mathcal{G} = \{x \mapsto a \operatorname{ReLU}(\operatorname{LN}_{\gamma, \beta}(wx + b)) + c : a, w, b, c, \gamma, \beta \in \mathbb{R}\}$$

$$d = 1 : \quad \mu(h) = h, \quad \sigma^2(h) = 0, \quad \hat{h} = 0, \quad \operatorname{LN}_{\gamma, \beta}(h) = \beta$$

Every LN network is constant, and both classes contain every constant function.

$$a = w = 1, \quad b = c = 0 \quad \implies \quad f(x) = \operatorname{ReLU}(x) = x \quad \text{for every } x \in [0, 1]$$

$$\mathcal{G} = \{x \mapsto k : k \in \mathbb{R}\} \subsetneq \mathcal{F} \quad \text{on } [0, 1]$$

A minimal counterexample, not a claim of strict loss for every wider network.

Summary: normalization and representation power

Greater representation power means a larger represented function class.

A larger class can lower the best training loss (representation error).

This best loss cannot increase, but may stay the same.

1. Motivation: use vector γ and β to increase representation power

2. In post-activation LN MLP, γ and β do not increase representation power

LN is right before an affine layer, which absorbs γ and β through W_2 and b_2 .

3. Pre-activation LN: γ and β can increase representation power

In the ReLU example, learned γ and β increase representation power.

4. LN (even with γ and β) may reduce representation power

Part 1.3

Gradients of LayerNorm

LayerNorm gradients: the main result and its use

How does LayerNorm change the backward error signal?

$$\tilde{z} = \text{LN}_{\gamma, \beta}(z), \quad J(z) = \frac{\partial \tilde{z}}{\partial z} \in \mathbb{R}^{d \times d}$$

J is the Jacobian of LN with respect to its input, with γ and β fixed.

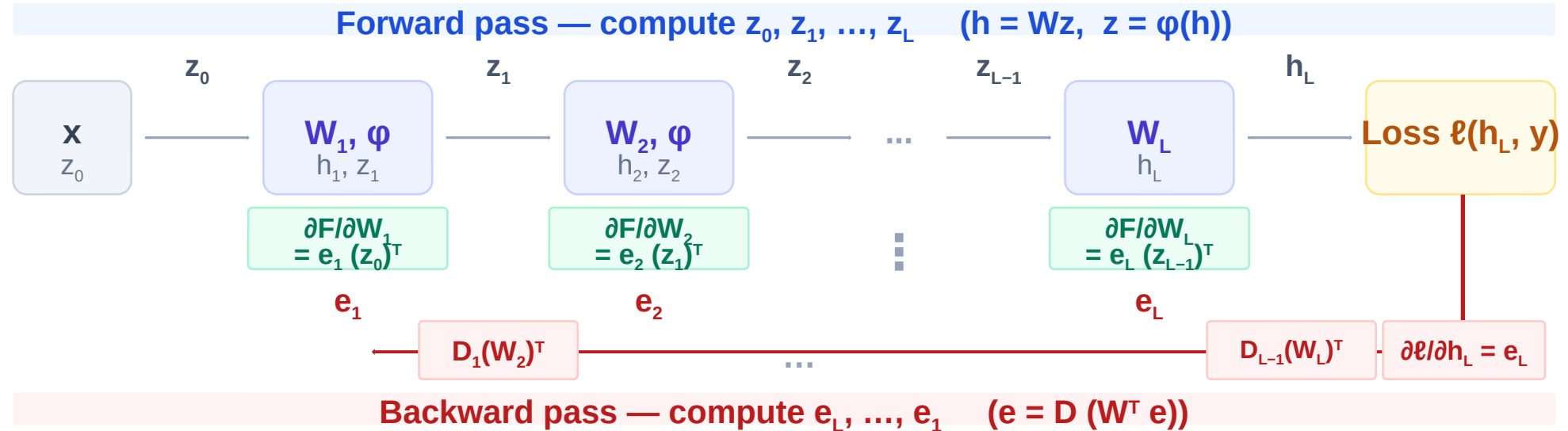
For column-vector loss gradients, the chain rule gives

$$\boxed{\frac{\partial F}{\partial z} = J(z)^\top \frac{\partial F}{\partial \tilde{z}}}$$

Recall Lecture 2, then insert J into one hidden layer and a general MLP.

Back-Propagation on the Computation Graph

One layer at a time. Forward walks left to right, backward right to left.



Forward: $h_i = W_i z_{i-1}, z_i = \phi(h_i)$. Backward: $e_i = D_i(W_{i+1})^T e_{i+1}$.

Lecture 2 recap: forward activations and backward errors

One sample, L weight matrices, $L - 1$ hidden layers, and no biases.

Forward pass: cache z and the diagonal entries of D

$$z_0 = x, \quad h_\ell = W_\ell z_{\ell-1}, \quad z_\ell = \phi(h_\ell) \quad (\ell = 1, \dots, L-1), \quad h_L = W_L z_{L-1}$$

$$F = \ell(h_L, y), \quad e_\ell = \nabla_{h_\ell} F, \quad D_\ell = \text{diag}(\phi'(h_\ell))$$

Backward pass: compute e from the output toward the input

$$e_L = \nabla_{h_L} \ell(h_L, y), \quad e_\ell = D_\ell W_{\ell+1}^\top e_{\ell+1} \quad (\ell = L-1, \dots, 1)$$

$$\underbrace{\nabla_{W_\ell} F}_{d_\ell \times d_{\ell-1}} = \underbrace{e_\ell}_{d_\ell \times 1} \underbrace{z_{\ell-1}^\top}_{1 \times d_{\ell-1}} \quad (\ell = 1, \dots, L)$$

Each weight gradient is an error vector times its input vector transposed.

Main result: the Jacobian of the LN operator

One sample $z \in \mathbb{R}^d$, with one learned γ and β per feature.

$$\mu = \frac{1}{d} \sum_i z_i, \quad \sigma^2 = \frac{1}{d} \sum_i (z_i - \mu)^2, \quad \hat{z} = \frac{z - \mu \mathbf{1}}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\tilde{z} = \text{LN}_{\gamma, \beta}(z) = \gamma \odot \hat{z} + \beta, \quad \gamma, \beta \in \mathbb{R}^d$$

$$J(z) = \frac{\partial \tilde{z}}{\partial z} = \frac{\text{diag}(\gamma)}{\sqrt{\sigma^2 + \varepsilon}} \left(I - \frac{\mathbf{1}\mathbf{1}^\top}{d} - \frac{\hat{z}\hat{z}^\top}{d} \right)$$

J has output coordinate j as its row and input coordinate i as its column.

$$\tilde{z}(z + \Delta z) = \tilde{z}(z) + J(z)\Delta z + o(\|\Delta z\|)$$

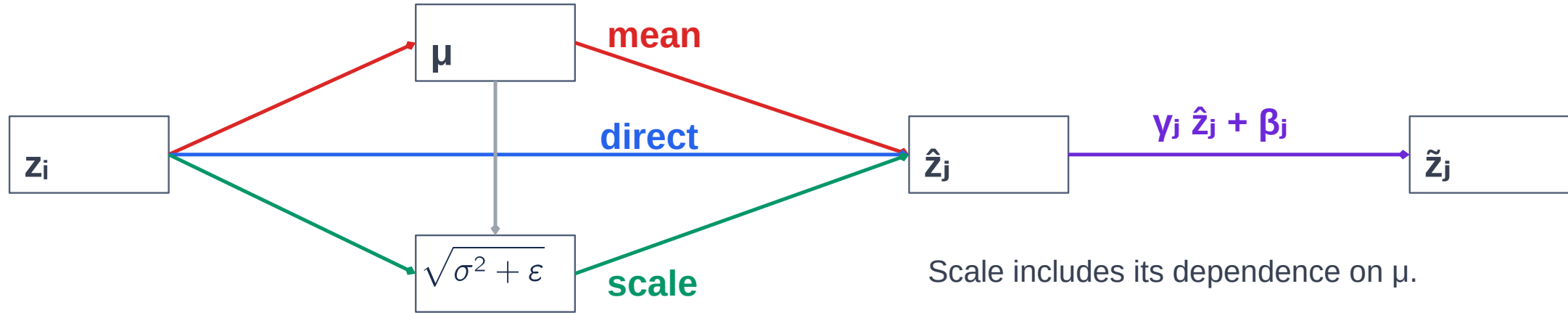
The local approximation describes changes: $\Delta \tilde{z} \approx J \Delta z$.

σ is the standard deviation. The denominator is $\sqrt{(\sigma^2 + \varepsilon)}$, with $\varepsilon > 0$.

LayerNorm: the chain rule adds three paths

We briefly explain how to calculate the Jacobian.

For one sample, fix input coordinate i and output coordinate j .



Multiply along each path, then add the three contributions.

$$\frac{\partial \hat{z}_j}{\partial z_i} = \underbrace{\frac{\delta_{ji}}{\sqrt{\sigma^2 + \epsilon}}}_{\text{direct}} - \underbrace{\frac{1}{d\sqrt{\sigma^2 + \epsilon}}}_{\text{mean}} - \underbrace{\frac{\hat{z}_j \hat{z}_i}{d\sqrt{\sigma^2 + \epsilon}}}_{\text{scale}}$$

$$J_{ji} = \gamma_j \frac{\partial \hat{z}_j}{\partial z_i} = \frac{\gamma_j}{\sqrt{\sigma^2 + \epsilon}} \left(\delta_{ji} - \frac{1}{d} - \frac{\hat{z}_j \hat{z}_i}{d} \right)$$

$\delta_{ji} = 1$ if $j = i$, and 0 otherwise. Detailed derivation: end-of-lecture appendix.

One hidden layer with LN: the gradient formulas

Use J to compute the weight gradients of a one-hidden-layer MLP with LN.
No weight-layer biases. LN retains featurewise γ_1 and β_1 .

$$h_1 = W_1 x, \quad z_1 = \phi(h_1), \quad \tilde{z}_1 = \text{LN}_{\gamma_1, \beta_1}(z_1), \quad h_2 = W_2 \tilde{z}_1$$



Backward: multiply by W_2^T , then J_1^T , then D_1 .

$$e_2 = \frac{\partial F}{\partial h_2}, \quad \boxed{e_1 = \frac{\partial F}{\partial h_1} = D_1 J_1^T W_2^T e_2}$$

W_2 takes $\tilde{z}_1$ as input; W_1 takes x as input.

$$\boxed{\nabla_{W_2} F = e_2 \tilde{z}_1^T}, \quad \boxed{\nabla_{W_1} F = e_1 x^T = D_1 J_1^T W_2^T e_2 x^T}$$

$D_1 = \text{diag}(\phi'(h_1))$. $J_1 = \partial \tilde{z}_1 / \partial z_1$ depends on $\hat{z}_1$, σ_1 and γ_1 (ϵ is fixed).

Backpropagation for an MLP with LN: the formulas

Generalize the one-hidden-layer example to L weight matrices (no weight-layer biases).

Forward pass

$$\tilde{z}_0 = x,$$

$$h_\ell = W_\ell \tilde{z}_{\ell-1},$$

$$z_\ell = \phi(h_\ell),$$

$$\tilde{z}_\ell = \text{LN}_{\gamma_\ell, \beta_\ell}(z_\ell) \quad (\ell < L),$$

$$h_L = W_L \tilde{z}_{L-1}, \quad F = \ell(h_L, y).$$

Two local derivative matrices

$$D_\ell = \text{diag}(\phi'(h_\ell))$$

$$J_\ell = \frac{\partial \tilde{z}_\ell}{\partial z_\ell} = \frac{\text{diag}(\gamma_\ell)}{\sqrt{\sigma_\ell^2 + \varepsilon}} \left(I - \frac{\mathbf{1}\mathbf{1}^\top}{d_\ell} - \frac{\hat{z}_\ell \hat{z}_\ell^\top}{d_\ell} \right)$$

$$\hat{z}_\ell = \frac{z_\ell - \mu_\ell \mathbf{1}}{\sqrt{\sigma_\ell^2 + \varepsilon}}, \quad \mu_\ell = \frac{1}{d_\ell} \sum_j z_{\ell,j}$$

$$\sigma_\ell^2 = \frac{1}{d_\ell} \sum_j (z_{\ell,j} - \mu_\ell)^2$$

Backward errors: e_ℓ is the gradient with respect to h_ℓ , before activation.

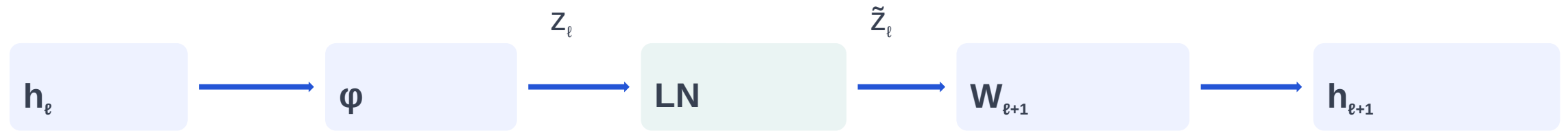
$$e_L = \frac{\partial F}{\partial h_L}, \quad \boxed{e_\ell = \frac{\partial F}{\partial h_\ell} = D_\ell J_\ell^\top W_{\ell+1}^\top e_{\ell+1}} \quad (\ell = L-1, \dots, 1)$$

Weight gradients: the input of W_ℓ is $\tilde{z}_{\ell-1}$

$$\boxed{\frac{\partial F}{\partial W_\ell} = e_\ell \tilde{z}_{\ell-1}^\top} \quad (\ell = 1, \dots, L)$$

Multiple hidden layers with LN: backward on the graph

One hidden layer of the computation graph



$D_\ell = \text{diag}(\varphi'(h_\ell))$. $J_\ell = \partial \tilde{z}_\ell / \partial z_\ell$ depends on $\hat{z}_\ell$, σ_ℓ and γ_ℓ (ϵ is fixed).

Backward pass: follow the operations from right to left

$$e_\ell \xleftarrow{D_\ell} \nabla_{z_\ell} F \xleftarrow{J_\ell^\top} \nabla_{\tilde{z}_\ell} F \xleftarrow{W_{\ell+1}^\top} e_{\ell+1}$$

$$\underbrace{e_\ell}_{d_\ell \times 1} = \underbrace{D_\ell}_{d_\ell \times d_\ell} \underbrace{J_\ell^\top}_{d_\ell \times d_\ell} \underbrace{W_{\ell+1}^\top}_{d_\ell \times d_{\ell+1}} \underbrace{e_{\ell+1}}_{d_{\ell+1} \times 1}$$

The order matters: $J_\ell^\top$ belongs between D_ℓ and $W_{\ell+1}^\top$.

Gradients of the learned γ and β

For one sample, compute the loss gradients for the two learnable parameter vectors.

Suppose $g = \partial F / \partial \tilde{z}$ is the incoming column-vector gradient of the scalar loss F .

$$\tilde{z}_j = \gamma_j \hat{z}_j + \beta_j, \quad j = 1, \dots, d$$

Gradient with respect to γ

$$\boxed{\frac{\partial F}{\partial \gamma} = g \odot \hat{z}}, \quad \frac{\partial F}{\partial \gamma_j} = g_j \hat{z}_j$$

Gradient with respect to β

$$\boxed{\frac{\partial F}{\partial \beta} = g}, \quad \frac{\partial F}{\partial \beta_j} = g_j$$

Each gradient has d entries and costs $O(d)$ to compute.

Storage and computation

So far: gradient formulas for optimization

We know which gradients to compute and how they propagate through the network.

Next: how do we implement those formulas efficiently?

The same gradient formula can require very different amounts of memory and computation. We now examine which values to save and how to evaluate the formulas efficiently.

1. What do we store during the forward pass for backpropagation?
2. How much computation does each backward operation require?
3. How do width d , depth L and batch size B change these costs?

We start with one sample, then scale up to a mini-batch.

Reminder: computation cost of basic operations

Before computing the cost of LN, recall these basic linear algebra operations.

Let u, v be length- d vectors and A, B be $d \times d$ matrices.

Count scalar additions and multiplications using the usual algorithms.

Operation	Output	Number of scalar operations	Cost
Addition: $u + v$	d -vector	d additions	$O(d)$
Elementwise: $u \odot v$	d -vector	d multiplications	$O(d)$
Inner product: $u^T v$	scalar	d multiplies + $(d - 1)$ adds	$O(d)$
Outer product: uv^T	$d \times d$ matrix	d^2 multiplications	$O(d^2)$
Matrix-vector: Av	d -vector	d inner products of length d	$O(d^2)$
Matrix-matrix: AB	$d \times d$ matrix	d^2 inner products of length d	$O(d^3)$

A full $d \times d$ Jacobian times a vector costs $O(d^2)$, not $O(d^3)$.

These counts describe computation work, not elapsed time on a GPU.

Two ways to compute $J^T g$

For one sample, g is the loss gradient at the LN output $\tilde{z}$.

$$g = \frac{\partial F}{\partial \tilde{z}} \in \mathbb{R}^d, \quad q = \gamma \odot g$$

Naive method: store the full $d \times d$ matrix J

Compute $J^T g$ by multiplying this matrix by the vector g .

Storage: d^2 scalars. Computation: $O(d^2)$.

Better method: use the structure of J

Store $\hat{z}$ (d entries) and σ (1 entry). The model already stores γ .

$$\begin{aligned} J^T g &= \frac{1}{\sqrt{\sigma^2 + \varepsilon}} \left(I - \frac{\mathbf{1}\mathbf{1}^T}{d} - \frac{\hat{z}\hat{z}^T}{d} \right) q \\ &= \frac{1}{\sqrt{\sigma^2 + \varepsilon}} \left(q - \mathbf{1} \frac{\mathbf{1}^T q}{d} - \hat{z} \frac{\hat{z}^T q}{d} \right). \end{aligned}$$

I is the $d \times d$ identity matrix, and $\mathbf{1}$ is the vector of d ones.

Storage: $d + 1$ scalars per sample. Computation: $O(d)$.

One sample: what do we store during the forward pass?

Setting: L weight matrices, $L - 1$ hidden LN layers, d neurons per layer.

In this setting, each W_ℓ is $d \times d$ and the output layer is linear.

Saved values at layer ℓ	Use in the backward pass	Scalars
$\tilde{z}_\ell$ and $\hat{z}_\ell$ (two vectors)	Weight input and LN gradients	$2d$
$\text{diag}(D_\ell) = \varphi'(h_\ell)$	Activation derivative	d
σ_ℓ	Scale in the LN derivative	1

One sufficient activation cache: $3d + 1$ scalars per sample per hidden layer.

Saving $\hat{z}_\ell$ and σ_ℓ makes μ_ℓ unnecessary in the backward formulas.

The chosen cache also makes raw h_ℓ and z_ℓ unnecessary. Store W , y and β separately.

One sample: computation cost with and without LN

Setting: each weight matrix is $d \times d$. Count scalar operations in one backward pass.

Backward operation at one layer	Computation cost
Weight transpose times an error vector	$O(d^2)$
LN product $J_\ell^T g$, using vector operations	$O(d)$
Activation product $D_\ell v$	$O(d)$
Form all entries of $e_\ell \tilde{z}_{\ell-1}^T$	$O(d^2)$

Why $O(d^2)$ for the outer product?

$$\left[\frac{\partial F}{\partial W_\ell} \right]_{ij} = (e_\ell)_i (\tilde{z}_{\ell-1})_j, \quad i, j = 1, \dots, d$$

Two vectors have $2d$ entries, but their outer product has d^2 entries.

Across L layers: $O(Ld^2)$ MLP backpropagation, with $O(Ld)$ additional work for LN.

One sample: activation and parameter storage

Illustrative MLP: $d = 100,000$, 96 hidden LN layers and $L = 97$ weight matrices.

Each W_ℓ is $d \times d$, without biases, using float32 (4 bytes per scalar).

Activation cache: one sample, across all 96 hidden layers

Saved activations	Number of scalars	Memory
All $\tilde{z}_\ell$ and $\hat{z}_\ell$ vectors	19,200,000	76.80 MB
All $\text{diag}(D_\ell)$ vectors	9,600,000	38.40 MB
All σ_ℓ values	96	384 bytes
Total activation cache	28,800,096	115.20 MB

Shared parameter storage: one copy for the whole model

Stored parameters	Number of scalars	Memory
All W_ℓ matrices	970,000,000,000	3.88 TB
All γ_ℓ and β_ℓ vectors	19,200,000	76.80 MB
Total parameter storage	970,019,200,000	3.88008 TB

LN's γ and β add only about 0.002% of the W -storage cost in this example.

Excludes gradients, optimizer states, input/loss caches and temporary buffers. Decimal MB/TB.

Mini-batches: activations grow, parameters stay fixed

Same MLP: $d = 100,000$, 96 LN layers, $L = 97$, float32, no recomputation.

Compare one sample with a mini-batch of $B = 8192$ samples.

Stored values	$B = 1$	$B = 8192$	Change
All $\tilde{z}_\ell$ and $\hat{z}_\ell$ vectors	76.80 MB	629.15 GB	8192×
All $\text{diag}(D_\ell)$ vectors	38.40 MB	314.57 GB	8192×
All σ_ℓ values	384 bytes	3.15 MB	8192×
Total activation cache	115.20 MB	943.72 GB	8192×
Total W, y and β storage	3.88008 TB	3.88008 TB	Unchanged

For B samples, save B sets of activations and one copy of W , y and β .

$M_{\text{act}}(B)$: number of activation scalars saved for a mini-batch of B samples.

M_{param} : number of scalars stored in all model parameters W , y and β .

$$M_{\text{act}}(B) = B(L - 1)(3d + 1), \quad M_{\text{param}} = Ld^2 + 2(L - 1)d$$

MLP backpropagation: $O(BLd^2)$, additional LN computation: $O(BLd)$

Counts are scalars, not bytes. Excludes gradients, optimizer states and temporary memory.

Changing the layer changes the backward implementation

MLP placement: the order of D and J changes

Post-activation LN

$$h \xrightarrow{\phi} z \xrightarrow{\text{LN}} \tilde{z}, \quad e = DJ^{\top} r$$

Pre-activation LN

$$h \xrightarrow{\text{LN}} q \xrightarrow{\phi} z, \quad e = J^{\top} Dr$$

D uses the input to ϕ . J uses the input to LN. r is the upstream error.

The forward values determine which derivatives we save

- After ϕ : compute D at h and compute J at $z = \phi(h)$.
- Before ϕ : compute J at h and compute D at $q = \text{LN}(h)$.
- Changing this order changes the inputs needed for the backward computation.

The following BatchNorm section will introduce a different normalization group.

History

Before leaving LayerNorm, we recall where the original paper placed it.

2016: Ba, Kiros & Hinton

The original LayerNorm uses pre-activation LN: normalize h before applying ϕ .

In our notation, with the bias omitted:

$$h = Wx, \quad z = \phi(\text{LN}(h))$$

h : pre-activation vector. LN includes the learnable γ and β .

Later: LayerNorm in Transformers

Transformers use more complex structures. We will discuss them later.

Ba, Kiros & Hinton (2016), Layer Normalization, Eq. (5).

Part 2

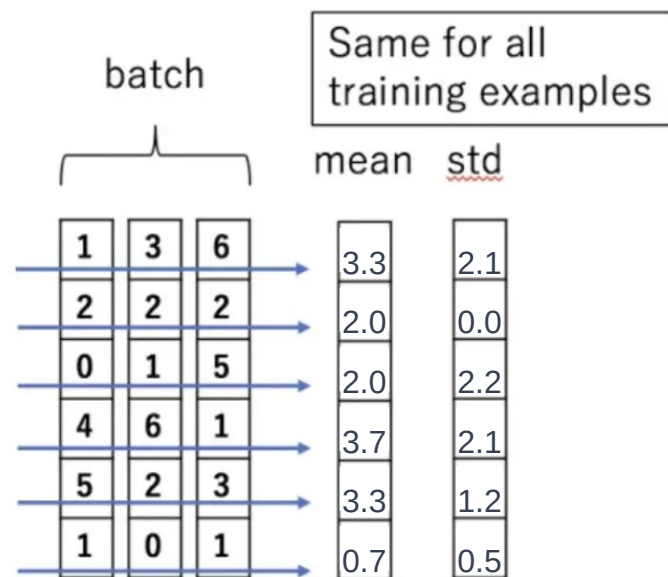
Batch normalization

Part 2.1

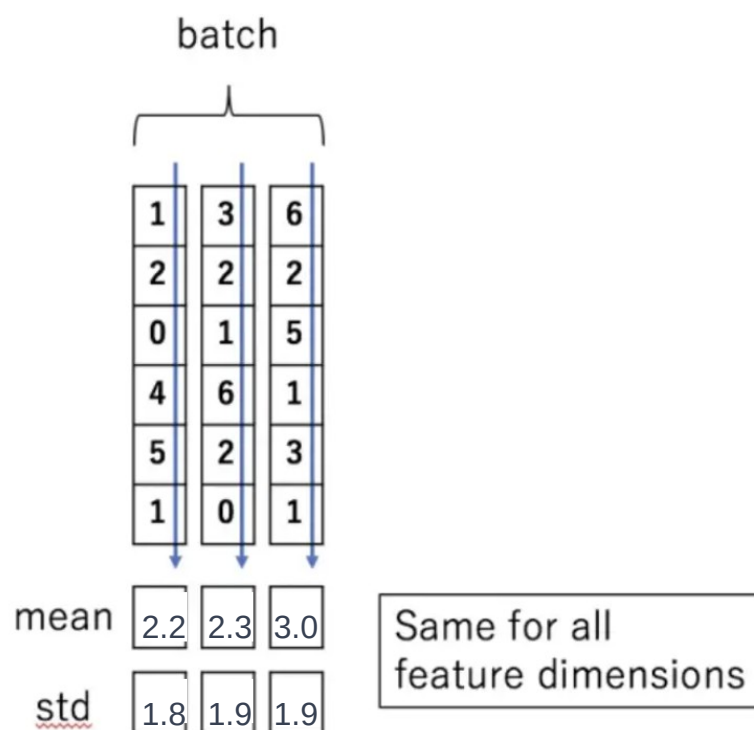
Introduction to BatchNorm

BatchNorm and LayerNorm: rows versus columns

Batch Normalization



Layer Normalization



BatchNorm

Across samples.
One mean and std per row.

Example: first row

$$\mu = 10/3, \sigma \approx 2.055$$

Input row / BN output

1	3	6
-1.136	-0.162	1.298

$$\gamma = 1, \beta = 0, \varepsilon = 0$$

LayerNorm

Across features.
One mean and std per column.

The figure rounds statistics to one decimal; the BN output uses the unrounded statistics.

BatchNorm: one row, one normalization group

Activation matrix $Z = Z_1$ (6×3)

Here $d = d_1 = 6$ and mini-batch size $B = 3$.

	Sample 1	Sample 2	Sample 3
Feature 1	Z_{11}	Z_{12}	Z_{13}
Feature 2	Z_{21}	Z_{22}	Z_{23}
Feature 3	Z_{31}	Z_{32}	Z_{33}
Feature 4	Z_{41}	Z_{42}	Z_{43}
Feature 5	Z_{51}	Z_{52}	Z_{53}
Feature 6	Z_{61}	Z_{62}	Z_{63}

Z_{ji} : feature j of sample i .

Highlighted row: $j = 1$.

The entries in one row share μ_j , σ_j , γ_j and β_j .

Fix feature j and sum over samples i

$$\mu_j = \frac{1}{B} \sum_{i=1}^B Z_{ji}$$

$$\sigma_j^2 = \frac{1}{B} \sum_{i=1}^B (Z_{ji} - \mu_j)^2$$

$$\tilde{Z}_{ji} = \gamma_j \frac{Z_{ji} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j$$

γ_j and β_j are scalars for feature j .

$\epsilon > 0$ is a fixed constant.

Quick question: NO-S or NO-V?

For one feature row, which operator does training-time BN use?

- A. NO-S: one scalar γ and β for the whole row.
- B. NO-V: independent γ and β entries for each sample.
- C. Neither.
- D. I do not know.

BN and LN: scalar or vector within one group?

Let Z have d feature rows and B sample columns.

BN: one feature across samples

$$u_j = (Z_{j1}, \dots, Z_{jB})^T \in \mathbb{R}^B$$

Standardize this row to obtain $\hat{u}_j$.

$$\tilde{u}_j = \gamma_j \hat{u}_j + \beta_j \mathbf{1}_B$$

One scalar γ_j and β_j for all B entries.

Different rows have their own scalar pairs.

LN: one sample across features

$$z^{(i)} = (Z_{1i}, \dots, Z_{di})^T \in \mathbb{R}^d$$

Standardize this column to obtain $\hat{z}^{(i)}$.

$$\tilde{z}^{(i)} = \gamma \odot \hat{z}^{(i)} + \beta$$

Vectors γ and β contain d entries each.

Every sample column uses the same vectors.

Across the whole layer, both standard BN and LN have d gains and d biases.

BatchNorm: one γ , β pair for each feature

One mini-batch: 3 samples, 6 features. Normalize each feature row across samples.

Post-activation matrix Z				Computed statistics		Learned pair	
	Sample 1	Sample 2	Sample 3		μ_j	σ_j	
Feature 1	1	3	6	→	3.3	2.1	γ_1, β_1
Feature 2	2	2	2	→	2.0	0.0	γ_2, β_2
Feature 3	0	1	5	→	2.0	2.2	γ_3, β_3
Feature 4	4	6	1	→	3.7	2.1	γ_4, β_4
Feature 5	5	2	3	→	3.3	1.2	γ_5, β_5
Feature 6	1	0	1	→	0.7	0.5	γ_6, β_6

The same pair applies to every sample in its row.

$$\tilde{Z}_{j,i} = \gamma_j \frac{Z_{j,i} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j, \quad i = 1, 2, 3.$$

BN and LN with 20 samples, 5 features

One hidden layer: $d = 5$ neurons, $B = 20$ samples, $Z \in \mathbb{R}^{5 \times 20}$.

Quantity in one mini-batch	Standard BN	Standard LN
1. Entries normalized together	5 rows of 20 samples	20 columns of 5 features
2. Computed μ (same count for σ)	_____	_____
3. y values in one normalized group	_____	_____
4. y values in the whole layer	_____	_____

Vote separately for each row. Every pair means (BN count, LN count).

Row 2 A. (5, 5) B. (5, 20) C. (20, 20) D. None of the above

Row 3 A. (1, 1) B. (5, 1) C. (1, 5) D. None of the above

Row 4 A. (5, 5) B. (20, 20) C. (5, 20) D. None of the above

One row for BN, one column for LN. The β counts are the same as the y counts.

BN and LN: twenty samples, five features

One hidden layer: $d = 5$ neurons, $B = 20$ samples, $Z \in \mathbb{R}^{5 \times 20}$.

Quantity in one mini-batch	Standard BN	Standard LN
Entries normalized together	5 rows of 20 samples	20 columns of 5 features
Computed statistics	5 μ and 5 σ	20 μ and 20 σ
γ and β in one normalized group	1 scalar γ_j and 1 scalar β_j	5 γ entries and 5 β entries
γ and β in the whole layer	$5 \gamma + 5 \beta = 10$	$5 \gamma + 5 \beta = 10$

$$\gamma = (\gamma_1, \dots, \gamma_5)^T, \quad \beta = (\beta_1, \dots, \beta_5)^T$$

With 100 samples and 5 features, both still use 5 γ values and 5 β values.

The index j on γ_j and β_j identifies a feature, not a sample or a mini-batch.

100 mini-batches: what changes?

One hidden layer: $d = 5$ neurons, $B = 20$ samples per mini-batch.

100 mini-batches $\times$ 20 samples per batch = 2,000 samples in one pass

Quantity	Standard BN	Standard LN
Computed μ and σ in one batch	5 μ and 5 σ	20 μ and 20 σ
Computed μ and σ in 100 batches	500 μ and 500 σ	2,000 μ and 2,000 σ
γ and β in the whole layer	$5 \gamma + 5 \beta = 10$	$5 \gamma + 5 \beta = 10$

The totals count computations over one pass, not stored parameter entries.

Every mini-batch uses the same 5 gains and 5 biases.

BN stores 5 running means and 5 running variances for ordinary inference.

LN stores no running statistics.

BN and LN: the same parameter count

Why do BN and LN use the same number of γ and β parameters?

1. Increase representation power

Use as many γ and β parameters as possible.

2. Share parameters across samples

Every sample uses the same γ and β vectors.

Affine parameters applied to $\hat{Z}$: rows are features, columns are samples.

	Sample 1	Sample 2	...	Sample B
Feature 1	γ_1, β_1	γ_1, β_1	...	γ_1, β_1
Feature 2	γ_2, β_2	γ_2, β_2	...	γ_2, β_2
...	...	...	...	...
Feature d	γ_d, β_d	γ_d, β_d	...	γ_d, β_d

Both use d γ values & d β values, regardless of the number of samples.

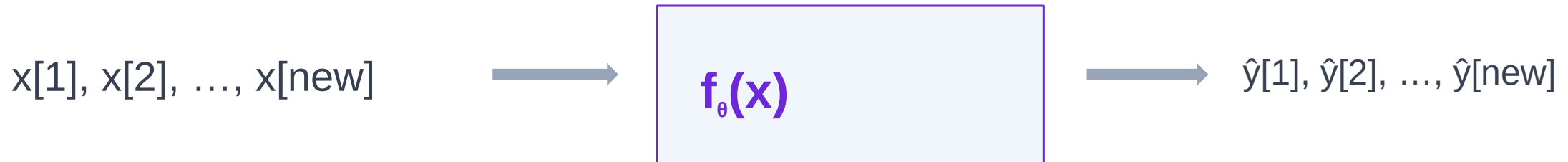
One parameter set for all samples and batches

Both BN and LN share one learned set of γ and β across all samples and mini-batches.

To understand why, consider prediction on new inputs.

At inference, arbitrarily many new samples may arrive.

The same trained network makes predictions for every input.



Fixed model parameters θ , including γ and β

The learned γ and β must work across samples, including inputs never seen in training.

BatchNorm on a matrix: NO-S on each feature row

Training-time BN: d feature rows and B sample columns.

$$Z \in \mathbb{R}^{d \times B}, \quad \gamma = (\gamma_1, \dots, \gamma_d)^\top, \quad \beta = (\beta_1, \dots, \beta_d)^\top$$

$$\text{BN}_{\gamma, \beta}(Z) \triangleq \begin{pmatrix} \text{NO-S}_{\gamma_1, \beta_1}(Z_{1,:}) \\ \text{NO-S}_{\gamma_2, \beta_2}(Z_{2,:}) \\ \vdots \\ \text{NO-S}_{\gamma_d, \beta_d}(Z_{d,:}) \end{pmatrix} \in \mathbb{R}^{d \times B}$$

$$\text{NO-S}_{\gamma_j, \beta_j}(Z_{j,:}) = \gamma_j \frac{Z_{j,:} - \mu_j \mathbf{1}_B^\top}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j \mathbf{1}_B^\top$$

μ_j and σ_j^2 are the mean and variance across the B entries of row j .

Scalar pair within each row; vectors γ and β across the d rows.

BatchNorm in a multilayer MLP

Recursive form

$$\tilde{Z}_0 = X$$

$$Z_l = \phi(W_l \tilde{Z}_{l-1} + b_l \mathbf{1}^T)$$

$$\tilde{Z}_l = \text{BN}_{\gamma_l, \beta_l}(Z_l)$$

$$\hat{Y} = W_L \tilde{Z}_{L-1} + b_L \mathbf{1}^T$$

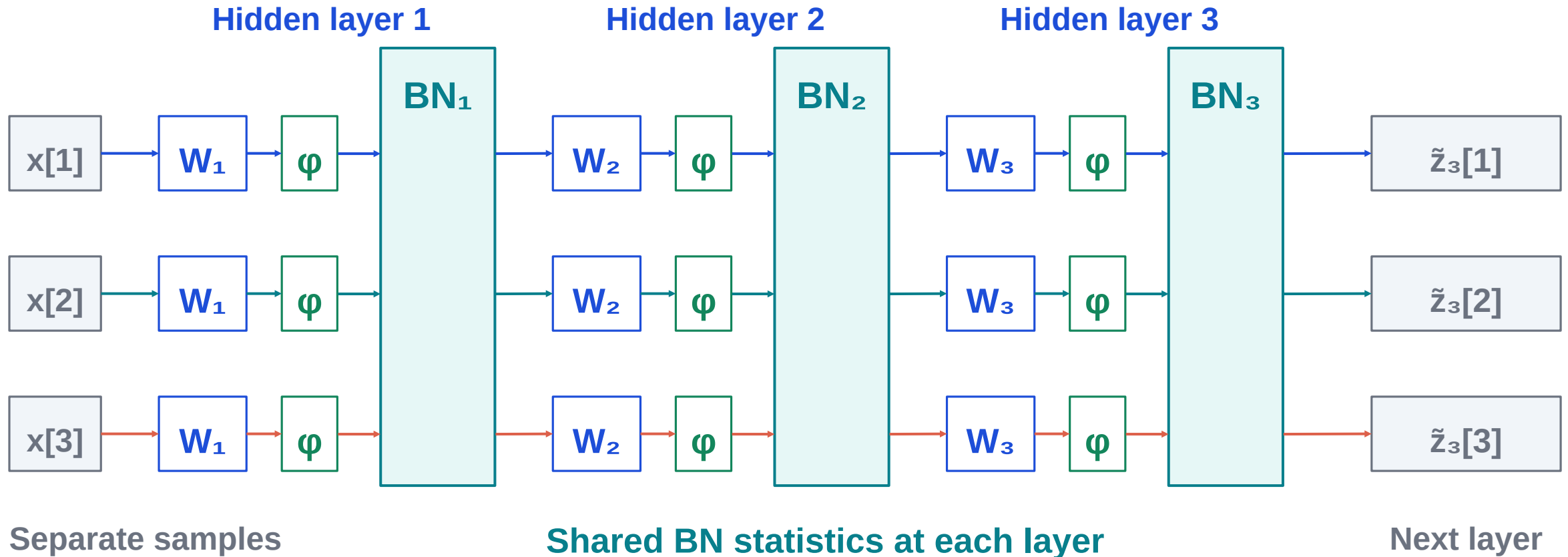
The network takes a batch jointly

- The matrix Z_ℓ contains one column per sample.
- Row j uses NO-S with scalar γ_j and β_j .
- The next layer takes $\tilde{Z}_\ell$ as its input.
- The trainable parameters include all W_ℓ , b_ℓ , γ_ℓ , and β_ℓ .

We use post-activation normalization throughout the MLP examples.

BatchNorm: shared statistics at every layer

During training, BN uses statistics across the samples at every hidden layer.



Each BN computes a separate mean and variance for every feature.

Training-time BN: the loss depends on the mini-batches

Fix T mini-batches of B samples each: $N = TB$ training samples.

θ : model parameters. $\mathcal{B}$: batch partition.

$x[k,i]$ and $y[k,i]$: input and target of sample i in mini-batch k .

$$X_k = [x[k, 1], \dots, x[k, B]], \quad k = 1, \dots, T$$

$$[\hat{y}[k, 1], \dots, \hat{y}[k, B]] = \tilde{f}_\theta(X_k)$$

The network takes B inputs jointly and produces B predictions.

$$F_{\mathcal{B}}(\theta) = \frac{1}{TB} \sum_{k=1}^T \sum_{i=1}^B \ell(\hat{y}[k, i], y[k, i]) = \frac{1}{T} \sum_{k=1}^T F_k(\theta)$$
$$F_k(\theta) = \frac{1}{B} \sum_{i=1}^B \ell(\hat{y}[k, i], y[k, i])$$

Each prediction $\hat{y}[k,i]$ can depend on every sample in batch k .

The loss decomposes over batches, not independent single-sample forward passes.

Changing the partition $\mathcal{B}$ can change the objective $F_{\mathcal{B}}$, even with the same data.

History of BatchNorm

Sergey Ioffe & Christian Szegedy introduced BatchNorm in 2015.

Original paper: pre-activation BN

$$Z = \varphi(\text{BN}(H))$$

This lecture: post-activation BN

$$Z = \varphi(H), \quad \tilde{Z} = \text{BN}(Z)$$

$H = WX$ is the pre-activation matrix for mini-batch X (bias omitted).

φ is the activation function. BN includes the learnable γ and β .

Studies of BN after the activation

Mishkin & Matas (2016): better in most tested FitNet settings on CIFAR-10.

Mishkin et al. (2016): better for CaffeNet on ImageNet, but not GoogLeNet.

The better placement depends on the architecture. We skip the details here.

Common choices, not universal rules

CNN image classifiers often use BN. Transformers commonly use LN or RMSNorm.

LN also works in image classifiers such as ViT and ConvNeXt.

Sources: BN §3.2; Mishkin studies, Tables 5 and 7; ViT; ConvNeXt; Llama 3.

Part 2.2

Training–inference consistency

—

Inference (validation; evaluation) vs. training

Machine learning pipeline: first, training; second, test / inference.

Training

Use an algorithm to update the parameters.

$$\theta^{\langle 0 \rangle} \longrightarrow \theta^{\langle 1 \rangle} \longrightarrow \dots \longrightarrow \theta^{\langle T \rangle}$$

This is also called “training the model”.

Inference

After training stops, fix the learned parameters.

$$x_{\text{test}} \longrightarrow f_{\theta^{\langle T \rangle}} \longrightarrow \hat{y}$$

$\theta^{\langle T \rangle}$ denotes the parameters after training.

Feed a test sample into the neural net and get an output as prediction.

For BN, fixing W , γ and β still leaves a question: where do μ and σ come from?

Notation: one feature across a mini-batch

For each mini-batch, the network generates post-activations:

$$\mathcal{B} = (x[1], \dots, x[B]), \quad Z_l \in \mathbb{R}^{d_l \times B}, \quad z_{l,j}[i] = (Z_l)_{j,i}$$

Mini-batch statistics: one mean and variance for each feature row

$$\mu_{\mathcal{B},l,j} = \frac{1}{B} \sum_{i=1}^B z_{l,j}[i], \quad \sigma_{\mathcal{B},l,j}^2 = \frac{1}{B} \sum_{i=1}^B (z_{l,j}[i] - \mu_{\mathcal{B},l,j})^2$$

To simplify the notation, focus on one layer and one feature.

$z[i], \mu_{\mathcal{B}}, \sigma_{\mathcal{B}}$ for one fixed layer l and feature j .

σ is the standard deviation; ϵ will be included in the normalization denominator.

Inference stage: major steps

SBN uses the scalar γ , β rule from NO-S, with supplied μ and σ .

$$\text{SBN}_{\gamma,\beta}(z; \mu, \sigma) = \gamma \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

Recall NO-S on a vector u :

$$[\text{NO-S}_{\gamma,\beta}(u)]_i = \text{SBN}_{\gamma,\beta}(u_i; \mu(u), \sigma(u))$$

NO-S computes statistics from u . SBN receives μ and σ as inputs.

First Compute or select μ and σ for each layer and each feature.

Second Replace BN by SBN using those statistics.

Third Apply the resulting network to each test sample.

Different inference methods differ in where μ and σ come from.

Inference Method 1: use the test mini-batch

Use the current test mini-batch to compute μ and σ .

Issue: prediction must wait for a full test mini-batch.

Its result also depends on the other samples in that batch.



Example: Airport face scanner

Suppose each test mini-batch needs 8 people.

The first traveler must wait for 7 more people.

With a test batch of 64: wait for 63 others.

This scenario illustrates why waiting for a full test batch may be impractical.

Illustration: N. Hanacek/NIST. The BN setting here is a teaching scenario.

Inference Method 2: use training-population statistics

Method 2: use the population statistics of the training data.

Motivation in the original BN paper

“we want the output to depend only on the input, deterministically.”

Ioffe & Szegedy (2015) · Batch Normalization

What are “population statistics”?

The mean and variance of features using the entire training dataset as one batch. [Wu & Johnson, 2021]

With fixed μ and σ , one test sample is enough.

Method 2: how do we compute population statistics?

Ideal method

- Feed all training data into the network with its parameters fixed.
- Record post-activations for all samples.
- Compute their mean and standard deviation.

Keeping all samples' activations in memory is costly.

Proposal: load one mini-batch at a time and accumulate statistics.

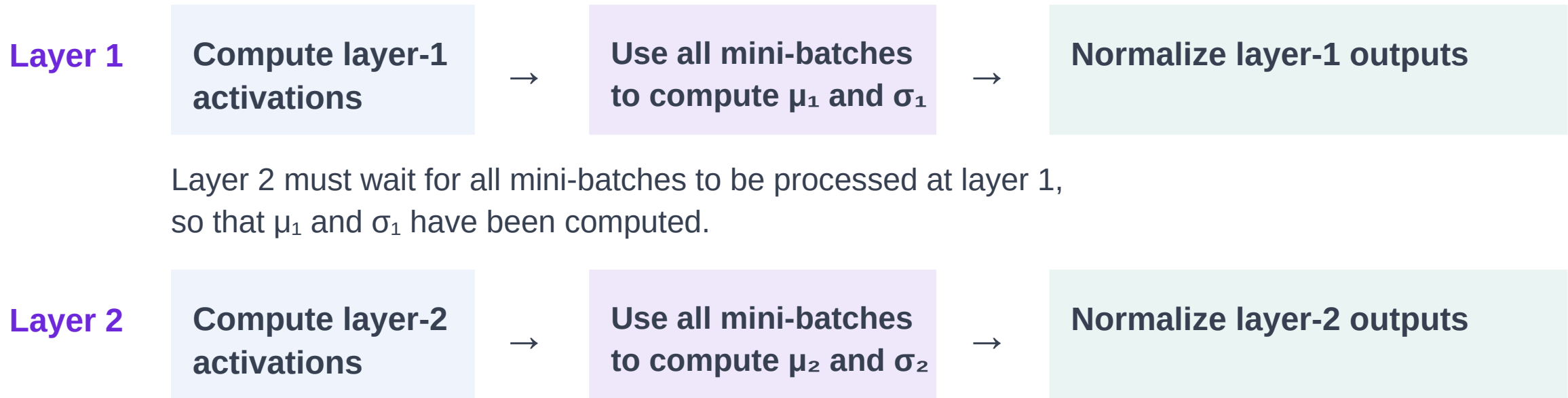
Remark: we have not studied gradient accumulation yet.

We may discuss this related idea later.

Can we do this in one pass through all mini-batches?

Exact population statistics: a naive method

Process all training mini-batches at one layer before moving to the next.



Repeat the full-dataset computation at every layer.

This naive exact method is impractical for large datasets and deep networks.

Wu & Johnson (2021), Appendix A.3: exact layer-by-layer computation.

Method 2.1: exponential moving averages (EMA)

How about approximation?

For each feature, maintain a running mean and variance during training.

$$\mu_{\text{EMA}} \leftarrow \lambda \mu_{\text{EMA}} + (1 - \lambda) \mu_{\mathcal{B}},$$

$$\sigma_{\text{EMA}}^2 \leftarrow \lambda \sigma_{\text{EMA}}^2 + (1 - \lambda) \sigma_{\mathcal{B}}^2.$$

- Update the estimates at each iteration using the current mini-batch.
- λ weights the old estimate, e.g. $\lambda = 0.9$.

BN moving-average estimates: Ioffe & Szegedy (2015), §3.1.

EMA equations and analysis: Wu & Johnson (2021), §3.1.

At inference, fix these estimates and keep the learned γ and β .

Issue of EMA

There are two notable issues. [Wu & Johnson, 2021]

1. Historical parameters

The stored statistics include features produced by earlier network parameters, not only by the final learned parameters.

2. Samples are not equally weighted

Recent mini-batches receive more weight than earlier mini-batches.

recent batch weights: $(1 - \lambda), \lambda(1 - \lambda), \lambda^2(1 - \lambda), \dots$

Can we compute the statistics using the current learned model instead?

Method 2.2: PreciseBN

Pick a subset of training samples and recompute μ and σ with the network parameters fixed after training.

- Keep W , γ and β fixed during this computation.
- Compute statistics on m mini-batches of the same size.
- Combine the statistics and store them for inference.

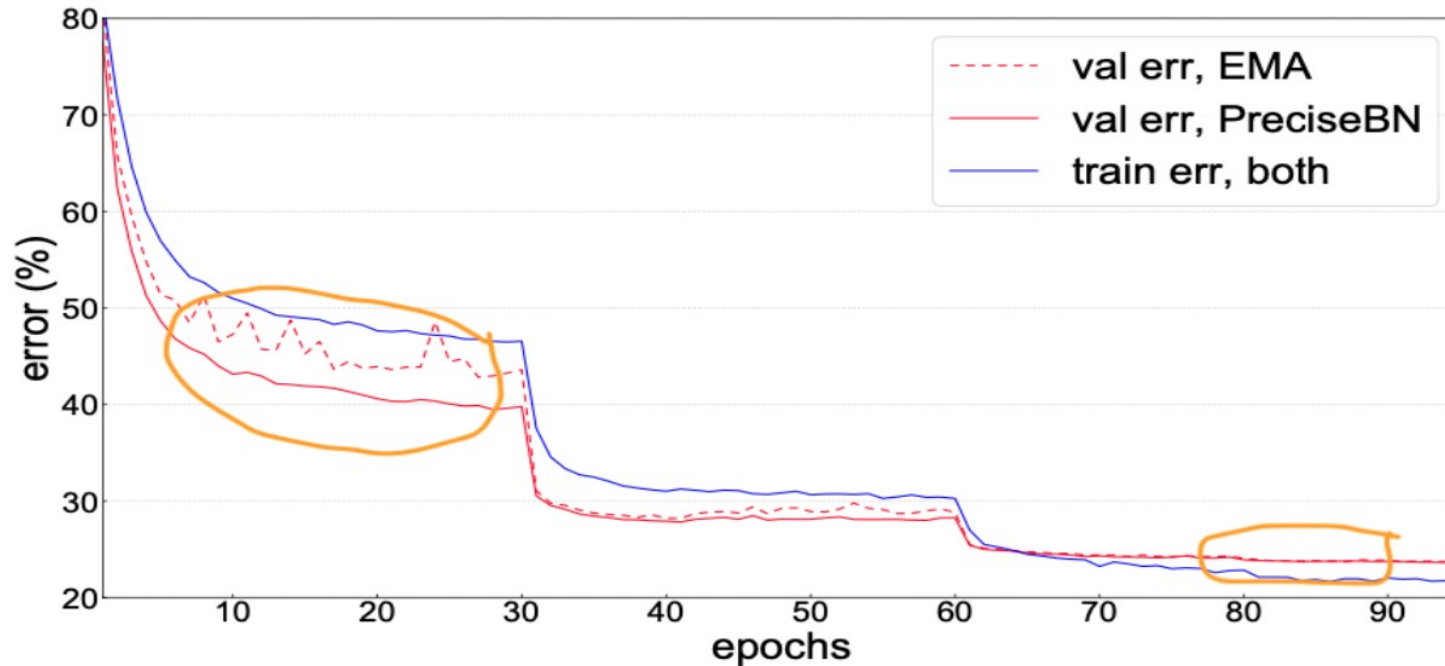
$$\hat{\mu}_{\text{pop}} = \frac{1}{m} \sum_{k=1}^m \mu_{\mathcal{B}_k}, \quad \hat{\sigma}_{\text{pop}}^2 = \frac{1}{m} \sum_{k=1}^m (\sigma_{\mathcal{B}_k}^2 + \mu_{\mathcal{B}_k}^2) - \hat{\mu}_{\text{pop}}^2$$

[Reading] The formula above is the alternative computation in the original lecture.

All selected samples use the same fixed network parameters.

Comparison of EMA and PreciseBN

ResNet-50 on ImageNet: training batch size 256, distributed on 8 GPUs.



Discovery in this experiment

- Better evaluation accuracy at early stages.
- More stable validation curves.

**Inference methods do not affect this training run,
so there is only one training-error curve.**

Original lecture figure · Wu & Johnson (2021), Fig. 3

Relevance: why should we care?

Compared with EMA, PreciseBN can give more reliable validation curves.

Possible implications

- Early validation fluctuations can come from inaccurate BN statistics.
- Comparing methods using early validation curves can therefore be misleading.

Before judging a trained model, check how its BN statistics were obtained.

LayerNorm: different training and test samples

One fixed checkpoint: $\gamma_j = 2$ and $\beta_j = 1$ for all three features.

Training: B = 2 samples

Inference: B = 1 new sample

Mode / sample	Input z ($d = 3$)	Its own statistics	LN output $\tilde{z}$
Training sample 1	(1, 2, 0)	$\mu = 1, \sigma^2 = 2/3$	(1, 3.449, -1.449)
Training sample 2	(2, 4, 6)	$\mu = 4, \sigma^2 = 8/3$	(-1.449, 1, 3.449)
New test sample	(4, 1, 1)	$\mu = 2, \sigma^2 = 2$	(3.828, -0.414, -0.414)

$$\tilde{z}_j = \gamma_j \frac{z_j - \mu}{\sigma_\epsilon} + \beta_j, \quad \sigma_\epsilon = \sqrt{\sigma^2 + \epsilon}$$

For this arithmetic example, $\epsilon = 0$ and every variance is positive.

Each sample supplies its own μ and σ across its $d = 3$ features.

LayerNorm: what is consistent between modes?

“Unlike batch normalization, layer normalization performs exactly the same computation at training and test times.”

Ba, Kiros & Hinton (2016), Layer Normalization, abstract

What LayerNorm computes

Use this sample’s feature mean and std.

Keep the trained γ and β for inference.

No running mean or variance is needed.

Why LN accepts a new test sample

Its own features supply μ and σ .

No test labels or other samples are needed.

A new input can have different statistics.

PyTorch LayerNorm documentation states:

“This layer uses statistics computed from input data in both training and evaluation modes.”

LN does not switch to stored statistics at inference.

A change in the test-data distribution can still affect prediction accuracy.

Summary: where do inference statistics come from?

All methods keep the learned γ and β .

Method	Source of μ and σ	Dependence at inference
BN Method 1	Current test mini-batch	Other test samples matter
BN Method 2.1: EMA	Running training estimates	Fixed for each checkpoint
BN Method 2.2: PreciseBN	Recomputed training estimates	Fixed for each checkpoint
Standard LN	Current sample's features	Only this sample is needed

PreciseBN improves estimation; BN still switches from batch to stored statistics.

LN uses the same per-sample rule in training and inference.

Practice questions: normalization axes and batch dependence

1. New network: Z_1 has shape 4×3

- What are d_1 and the mini-batch size B ?
- How many means does BN compute?
- How many does LN compute?
- How many values enter each mean?

2. Learned parameters

- With 4 features and 3 samples, how many γ and β values does BN learn?

3. Keep one sample unchanged

- Replace the other samples in its batch.
- Can its training-time BN output change?
- What about LN?

4. Inference

- Why use stored training statistics instead of estimating vector-BN statistics from a single test sample?

Explain the grouping and dependence, not only the names of the layers.

Lecture 4 summary: Normalization II

1. Computed statistics and learned parameters

- Standard LN learns one γ and β per feature, shared across samples.

2. Representation power

- Equal function classes imply equal minimum training loss (representation error).
- LN (even with γ and β) may reduce representation power.

3. Backpropagation through LayerNorm

- The backward error gains a factor J_ℓ^T between D_ℓ and $W_{\ell+1}^T$.

4. Batch normalization

- Compared with LN, BN normalizes across samples, not across features.
- Training–inference mismatch: batch statistics versus stored statistics.

References

Ioffe & Szegedy (2015)

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML.

Ba, Kiros & Hinton (2016)

Layer Normalization.

Wu & Johnson (2021)

Rethinking “Batch” in BatchNorm.

Mishkin & Matas (2016)

All you need is a good init. ICLR.

Mishkin, Sergievskiy & Matas (2016)

Systematic evaluation of CNN advances on the ImageNet. arXiv:1606.02228.

Implementation documentation

PyTorch: LayerNorm and BatchNorm1d.

Appendix A

Solutions to Questions in the Lectures

Solutions: LN parameter-counting questions

B = 5 samples, four hidden layers, eight neurons per layer

Each hidden layer has one separate LN module.

The third-layer activation matrix Z_3 has shape 8×5 .

Affine convention	Third hidden layer	All four hidden layers
Shared scalar (simplified)	$1 \gamma + 1 \beta = 2$ scalars	$4 \gamma + 4 \beta = 8$ scalars
Featurewise (standard LN)	$8 \gamma + 8 \beta = 16$ scalars	$32 \gamma + 32 \beta = 64$ scalars

- All five samples reuse the same parameters at a given layer.
- Doubling B leaves both parameter counts unchanged.

Computed statistics are separate: layer 3 has five μ values and five σ values.

Practice questions: solutions

Normalization axes and batch dependence

1. $d_1 = 4$ neurons and $B = 3$ samples.

BN: 4 means, 3 values per mean.

LN: 3 means, 4 values per mean.

2. BN learns 4 γ values and 4 β values, shared across all samples and mini-batches.

3. Training-time BN may change; LN does not depend on the other samples.

4. With one vector sample, current-batch statistics give zero centered values.

Fixed training statistics avoid this and remove test-batch dependence.

Appendix B

Details of Gradient Computation of LN

Recall: the LN Jacobian and backpropagation

Recall the LN gradient formulas from Part 1.3 (pages 34–37).

$$\mu = \frac{1}{d} \sum_i z_i, \quad \sigma^2 = \frac{1}{d} \sum_i (z_i - \mu)^2, \quad \hat{z} = \frac{z - \mu \mathbf{1}}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\tilde{z} = \text{LN}_{\gamma, \beta}(z) = \gamma \odot \hat{z} + \beta, \quad \gamma, \beta \in \mathbb{R}^d$$

$$J(z) = \frac{\partial \tilde{z}}{\partial z} = \frac{\text{diag}(\gamma)}{\sqrt{\sigma^2 + \varepsilon}} \left(I - \frac{\mathbf{1}\mathbf{1}^\top}{d} - \frac{\hat{z}\hat{z}^\top}{d} \right)$$

The chain rule propagates the loss gradient through LN:

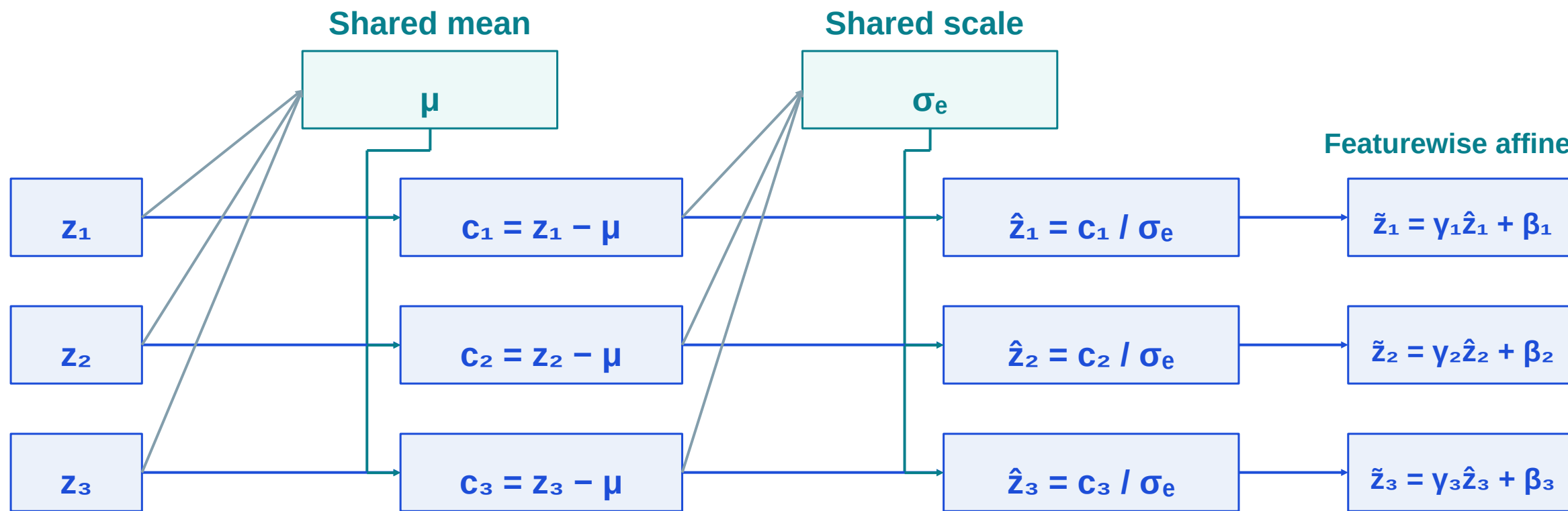
$$\frac{\partial F}{\partial z} = J(z)^\top \frac{\partial F}{\partial \tilde{z}}$$

We now derive J using the LN computation graph.

Notation in this appendix: $\sigma_e = \sqrt{(\sigma^2 + \varepsilon)}$, the same denominator as above.

LayerNorm: three neurons, one pair of statistics

One sample, three post-activation entries z_1, z_2, z_3 .



$$\mu = \frac{z_1 + z_2 + z_3}{3}$$

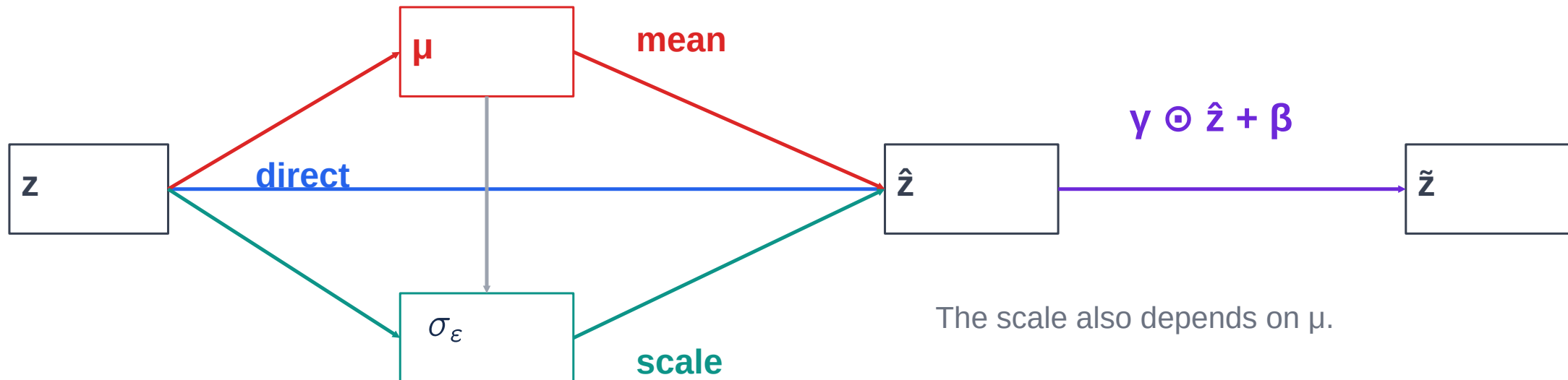
$$\sigma^2 = \frac{c_1^2 + c_2^2 + c_3^2}{3}, \quad \sigma_e = \sqrt{\sigma^2 + \epsilon}$$

σ = standard deviation; σ_e = stabilized scale.

One learned γ_j and β_j per feature.

The computation graph separates two steps

One sample: z has $d = d_\ell$ features.



$$\mu = \frac{1}{d} \sum_k z_k, \quad c = z - \mu \mathbf{1}, \quad \sigma_\epsilon = \sqrt{\frac{1}{d} \sum_k c_k^2 + \epsilon}, \quad \hat{z} = \frac{c}{\sigma_\epsilon}$$

$$\frac{\partial \tilde{z}}{\partial z} = \frac{\partial \tilde{z}}{\partial \hat{z}} \frac{\partial \hat{z}}{\partial z} \qquad \tilde{z} = \gamma \odot \hat{z} + \beta$$

Standardization: the chain rule adds three paths

Fix input coordinate i and output coordinate j .

Direct



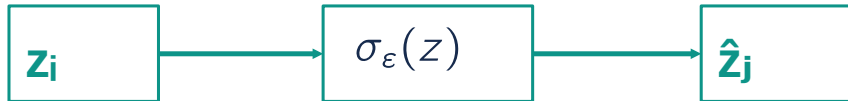
$$\left. \frac{\partial \hat{z}_j}{\partial z_i} \right|_{\mu, \sigma_\epsilon}$$

Via the mean



$$\frac{\partial \hat{z}_j}{\partial \mu} \frac{\partial \mu}{\partial z_i}$$

Via the scale



$$\frac{\partial \hat{z}_j}{\partial \sigma_\epsilon} \frac{d\sigma_\epsilon(z)}{dz_i}$$

Multiply derivatives along each path, then add the three contributions.

The scale derivative includes the dependence of μ on z .

The direct and mean contributions

Direct path: z_i to $\hat{z}_j$

$$\left. \frac{\partial \hat{z}_j}{\partial z_i} \right|_{\mu, \sigma_\epsilon} = \frac{\delta_{ji}}{\sigma_\epsilon}, \quad \delta_{ji} = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases}$$

Hold μ and σ_ϵ fixed for this partial derivative.

Mean path: z_i to μ to $\hat{z}_j$

$$\frac{\partial \hat{z}_j}{\partial \mu} = -\frac{1}{\sigma_\epsilon}, \quad \frac{\partial \mu}{\partial z_i} = \frac{1}{d} \quad \implies \quad -\frac{1}{d\sigma_\epsilon}$$

Every feature changes the shared mean by $1/d$.

Even when $i \neq j$, the mean contribution remains.

The scale path includes the mean dependency

$$\sigma^2 = \frac{1}{d} \sum_k c_k^2, \quad c_k = z_k - \mu, \quad \frac{\partial c_k}{\partial z_i} = \delta_{ki} - \frac{1}{d}$$

Differentiate the variance through all centered features

$$\frac{\partial \sigma^2}{\partial z_i} = \frac{2}{d} \sum_k c_k \left(\delta_{ki} - \frac{1}{d} \right) = \frac{2c_i}{d} - \frac{2}{d^2} \underbrace{\sum_k c_k}_0 = \frac{2c_i}{d}$$

The mean contribution cancels because the centered features sum to zero.

Differentiate the standard deviation

$$\frac{d\sigma_\epsilon}{dz_i} = \frac{1}{2\sigma_\epsilon} \frac{\partial \sigma^2}{\partial z_i} = \frac{c_i}{d\sigma_\epsilon}$$

$$\frac{\partial \hat{z}_j}{\partial \sigma_\epsilon} \frac{d\sigma_\epsilon}{dz_i} = \left(-\frac{c_j}{\sigma_\epsilon^2} \right) \frac{c_i}{d\sigma_\epsilon} = -\frac{c_j c_i}{d\sigma_\epsilon^3}$$

Derivation: the three paths give the full Jacobian J

First add the paths for standardization.

$$\frac{\partial \hat{z}_j}{\partial z_i} = \underbrace{\frac{\delta_{ji}}{\sigma_\epsilon}}_{\text{direct}} - \underbrace{\frac{1}{d\sigma_\epsilon}}_{\text{mean}} - \underbrace{\frac{c_j c_i}{d\sigma_\epsilon^3}}_{\text{scale}}, \quad c = z - \mu \mathbf{1}$$

Then differentiate the featurewise affine step.

$$J_{ji} = \frac{\partial \tilde{z}_j}{\partial z_i} = \gamma_j \frac{\partial \hat{z}_j}{\partial z_i} = \frac{\gamma_j}{\sigma_\epsilon} \left(\delta_{ji} - \frac{1}{d} - \frac{\hat{z}_j \hat{z}_i}{d} \right)$$

$$J(z) = \frac{\partial \tilde{z}}{\partial z} = \frac{\text{diag}(\gamma)}{\sigma_\epsilon} \left(I - \frac{\mathbf{1}\mathbf{1}^\top}{d} - \frac{\hat{z}\hat{z}^\top}{d} \right)$$

This is the same J used in the network gradient formulas.